# Compiler

## 参考编译器介绍

参考了课程组给出的sysy语言的编译器。

**总体结构**：编译器首先分为三部分：`src/`为源码部分，`test/`为测试部分，`tools/`为编译器开发和测试的辅助工具。

`src/`中`Compiler`为主类，在这里调用了词法分析、语法分析、生成中间代码与优化，最后输出结果。`front/`中包含了词法分析`lexer/`，语法分析`parser/`与`ast/`，前端于是可以完成词法分析语法分析，生成抽象语法树。`midend/`中完成了对于中间代码的生成。`backend/`中把中间代码翻译为目标mips代码。`error/`处理了各个阶段的错误，定义了所有的错误类型。`optimize/`中实现了各阶段需要的优化。`utils/`中有处理IO的和计算竞速复杂度的类。

**接口设计**：`Compiler`主类串联了各个阶段

`Frontend`：

- SetInput()：设置输入流。
- GenerateTokenList()：生成Token列表。
- GenerateAstTree()：生成AST语法树。
- GetTokenList()：获取Token列表。
- GetAstTree()：获取AST树。

`Midend`：

- GenerateSymbolTable()：生成符号表。
- GenerateIr()：生成IR中间代码。
- GetSymbolTable()：获取符号表。
- GetIrModule()：获取IR模块。

`Backend`：

- GenerateMips()：生成MIPS代码。

`Optimize`：

- Init()：初始化优化器。
- Optimize()：执行优化。

`utils`：

- IOhandler：负责输入输出的设置与打印。
- ErrorRecorder：错误记录与查询。

**文件组织**：`src/`为源码部分，`test/`为测试部分，`tools/`为编译器开发和测试的辅助工具。

`src/`中:

`frontend/`:前端模块，包含词法分析、语法分析、AST相关代码。子包：`lexer/`、`parser/`、`ast/`.

`midend`:`symbol/`（符号表）、`llvm/`（IR相关）、`visit/`（遍历与分析）

`backend`:BackEnd.java、MipsBuilder.java等、PeepHole.java（后端优化）

`utils`：`IOhandler.java`（输入输出）、`Setting.java`（配置）、`HandleComplexity.java`（复杂度分析）

`Compiler.java`:编译器主控类，包含主方法，负责串联各模块。

# 编译器总体设计

**总体结构**：

分为前中后端三部分，同时也设计优化部分，对全流程进行优化。前端完成词法语法分析。中端完成语义分析与中间代码生成。后端完成mips代码生成。工具来处理输入输出与错误处理还有配置管理等等。

**接口设计**：

前端给出GenerateTokenList()：生成Token列表，GenerateAstTree()：生成AST语法树。

中端给出GenerateIr()：生成IR中间代码。

后端给出GenerateMips()：生成MIPS代码。

优化：Init()：初始化优化器。Optimize()：执行优化。

错误处理：ErrorRecorder：错误记录与查询

**文件组织**：

`src/`为源码部分，`test/`为测试部分，`tools/`为编译器开发和测试的辅助工具。

`src/`中:

`frontend/`: 子包：`lexer/`、`parser/`、`ast/`，包含词法分析、语法分析、AST相关代码。

`midend`：符号表、IR相关代码

`backend`: mips汇编生成相关代码

`utils`：`IOhandler.java`（输入输出处理程序）、`Setting.java`（配置）、`HandleComplexity.java`（复杂度分析）

`Compiler.java`:编译器主控类，包含主方法，串联各模块。

# 词法分析设计

## 编码前设计

先建立Token类，作为词法分析的结果，属性包括token类型，token文本，行数。

因为token有很多类型，所以要建立一个对应关系，可以用java的给枚举类加属性的操作来实现。

还要读入文件把文件中内容转化为一个大的字符串。

然后就可以搭建有限自动机，用双指针，根据读到的内容不断推进，从而分析出各个单词。这里需要搭建多个有限自动机，然后共用一个相同的起始状态，把他们整理到一个大的有限自动机中。其中对于空白字符，先行指针跳过，然后到达非空白字符时停下，把先行赋值给后行指针。

然后得到的结果可以存在一个token的列表中，从而可以一个一个输出到文件中。

另外对于错误处理，仅需处理单个&或者|，因此包含在有限自动机的处理过程中了；但是为了方便后续编写，要建立错误类Error，并且把第几类这个信息包含进去，作为一个属性。

还要建立保留字表来确定是标识符还是保留字，这里可以建立一个映射，保留字的String与TokenType。

## 编码后修改设计

目前是有三个包，frontend，error，utils与不在包中的主类Compiler。

对于保留字，直接在Lexer类中写一个Map，最简单地实现保留字的映射。

对于Error类，学习教程中的给枚举类添加属性的操作，建立ErrorType类，实现了错误具体信息和类别的关系；在判断有没有错误来决定输出token还是错误的时候，根据一个全局的错误list来判断、实现。

对于文件读写设置了一个工具类FileProcess，在其中实现初始化输入输出与输入输出，还有关闭文件等等，其中要注意的是写文件还要flush。

在实现的细节上，我没有处理空白的跳过，是在for循环中自动跳过了。